Slack Integration Guide - PinionAI Agent Runner

This guide covers Slack app setup, local development, and production deployment of the Slack bot implemented in `chat_slack.py`.

Table of Contents

---

1. Slack App Configuration

To use the Slack integration, create and configure a Slack App in the Slack API Dashboard.

A. Create the App

1. Click **Create New App** > **From scratch**.
2. Name your app (for example, "PinionAI Bot") and select your workspace.

B. Enable Socket Mode

1. In the left sidebar, go to **Settings > Socket Mode**.
2. Toggle **Enable Socket Mode** to **On**.
3. Generate an **App-level token** with the `connections:write` scope.
4. Copy this token as `SLACK_APP_TOKEN`.
5. Enable **Interactivity & Shortcuts**, **Slash Commands**, and **Events**.

C. Configure Bot Scopes

1. Go to **Features > OAuth & Permissions**.
2. Add these bot scopes:
    - `chat:write`
    - `files:read`
    - `im:history`
    - `channels:history`
    - `groups:history`
3. Install the app to your workspace.
4. Copy the **Bot User OAuth Token** as `SLACK_BOT_TOKEN`.

D. Enable Events

1. Go to **Features > Event Subscriptions**.
2. Enable events.

3. Subscribe to:
    - `message.channels`
    - `message.groups`
    - `message.im`

E. Enable the Messages Tab

1. Go to **Features > App Home**.
2. Ensure **Messages Tab** is enabled.
3. Allow users to send slash commands and messages from the Messages tab.

F. Add Slash Commands

1. Go to **Features > Slash Commands**.
2. Create `/end` with a short description such as "Ends the PinionAI chat session".

---

2. Environment Setup

Use a local `.env` file for development and a server-side env file or Docker env file for production.

Example local environment:

```
client_id=<YOUR_CLIENT_ID>
client_secret=<YOUR_CLIENT_SECRET>
agent_id=<YOUR_AGENT_ID>
host_url=https://api.pinionai.com

SLACK_BOT_TOKEN=xoxb-...
SLACK_APP_TOKEN=xapp-...
```

For production, the same values can be placed in `deploy/prod-slack/env.yaml` for Cloud Run-style deployments, or converted into a `.env` file for a VM deployment.

---

3. Local Deployment

1. Activate your environment:

    ```
    source pinionai-streamlit/bin/activate
    ```

2. Install dependencies:

    ```
    uv pip compile requirements.in -o requirements.txt
    uv pip sync requirements.txt
    ```

3. Run the bot:

```
python chat_slack.py
```

---

4. Production Deployment on a VM

A VM is the recommended production hosting option for this Slack bot because it keeps a long-running Socket Mode connection alive.

4.1 Build the Slack-specific container image

Use the Slack-specific Dockerfile:

```
docker build -f Dockerfile.slack -t pinionai-slack:latest .
```

4.2 Prepare the environment file

On the VM, create a `.env` file with the same values you would normally put in `deploy/prod-slack/env.yaml`.

Example:

```
host_url=https://api.pinionai.com
client_id=<YOUR_CLIENT_ID>
client_secret=<YOUR_CLIENT_SECRET>
agent_id=<YOUR_AGENT_ID>
SLACK_BOT_TOKEN=xoxb-...
SLACK_APP_TOKEN=xapp-...
```

4.3 Run the container on the VM

```
docker run -d \
  --name pinionai-slack \
  --restart unless-stopped \
  --env-file /path/to/.env \
  pinionai-slack:latest
```

Check logs with:

```
docker logs -f pinionai-slack
```

4.4 Google Compute Engine VM

1. Create a Debian 12 or Ubuntu 22.04 VM in GCP.
2. Allow SSH and optionally HTTP/HTTPS traffic.
3. Install Docker on the VM.
4. Copy the repository to the server.
5. Build the image using `Dockerfile.slack`.
6. Start the container as shown above.

Example install steps on Debian/Ubuntu:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker "$USER"
```

4.5 DigitalOcean Droplet

1. Create a Debian 12 or Ubuntu 22.04 droplet.
2. SSH into the droplet.
3. Install Docker.
4. Clone the repository and build the Slack image.
5. Run the container with your `.env` file.

Example:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
```

4.6 Optional systemd service

If you prefer process management over Docker, run the bot directly from a Python virtual environment with a systemd unit. This is a good fit for long-running production use.

---

5. Usage Guide

Chatting

- **Direct Message:** Send a message to the bot in its Messages tab.
- **Channels:** Invite the bot to a channel and type a message.

Dynamic Agent Loading

You can change the active agent for a channel by uploading a `.aia` file.

1. Upload the `.aia` file to the chat.
2. The bot will switch to that agent for subsequent messages in that channel.
3. If the file is private, the bot will ask for the `key_secret`.

Cleaning Slack Formatting

The bot removes Slack formatting wrappers such as `<mailto:...>` before sending text to the AI.

Ending a Session

To clear the active session:

- Use `/end`, or
- Send `!end` as a message.

---

6. Troubleshooting

- **"Sending messages to this app has been turned off"**: Enable the Messages tab and user messaging in App Home.
- **Bot does not reply in channels**: Ensure the bot is invited to the channel and that `message.channels` is subscribed.
- **Bot does not see messages**: Check event subscriptions and app scopes.
- **Module import errors**: Make sure the virtual environment is activated and that dependencies are installed.